SESSION 1

Orientation & Complexity Analysis

Atharva Badve

01

Introduction

02

What we'll cover

Topics we will try to cover over the period of the course.

1. DSA
2. Operating Systems
3. Computer Architecture
4. Basics of System Software

03

Why this course, in the era of AI?

Fair question. Let's answer it properly.

“If AI can write the code, why should I learn to?”

Think about it for a minute. What would you say?

“If AI can write the code, why should I learn to?”

Fair question. It deserves a real answer, not a lecture about discipline.

1

You can't review what you can't write

2

Interviews test judgement, not typing

3

Fundamentals cannot be replaced.

04

Prerequisites

QUICK CHECK-IN

Tell us where you stand



How to fill it in

1. Scan the QR code or open the link

2. Takes about 5 minutes

3. Will be helpful for us to design future sessions

Form link: https://forms.gle/38Wg8wP9gtE2eJTQA

INSTALL BEFORE NEXT SESSION

Setup



1. Operating system

Windows, Linux, or Mac — whichever you already use.



2. Compiler

Latest compiler for your preferred language. Recommended: C/C++, Java, or Rust.



3. Editor

VS Code, Vim or anything else — your pick.



4. Debugger

The debugger for your preferred language (gdb/lldb, jdb, or the IDE's built-in one).



5. LeetCode account



6. GitHub account

YOUR REPOSITORY

How submissions work

```
interview-prep/
  README.md          <- name, language
  harness.hpp        <- shared test runner
  week01/
    01_dynamic_array.cpp
    02_kadane.cpp
    03_rotate.cpp
    notes.md         <- Explore notes
  week02/
    ...
```

1. **Public or private - your call**
   Private is fine. Add me as a collaborator.

2. **One folder per week**
   Numbered files so I can find things. Names as in the assignment.

3. **Commit as you go**
   Ten small commits beat one giant one.

4. **Deadline: before the next session**
   Not submitted to GitHub = not done.

05

Complexity analysis

The only question that matters, and how to answer it without running the code.

“It takes 0.2 seconds for n = 1,000. How long for n = 1,000,000?”

Take a guess. How would you find out without running it?

“It takes 0.2 seconds for n = 1,000. How long for n = 1,000,000?”

That is the entire reason complexity analysis exists.

Run it and wait

Could take 200 seconds. Could take 2 days. You won't know which until it finishes - or doesn't.

Count the work

Figure out how the number of operations grows with n. Then you know before you run anything.

THE IDEA

From counting to Complexity in three steps

```
int sum = 0;                // 1
for (int i = 0; i < n; i++)
    sum += a[i];            // n times
return sum;                 // 1
```

Roughly 3n + 2 operations, depending on how you count.

1 **Count** `3n + 2`

Every line runs some number of times. Add them up. The exact number does not matter much.

2 **Drop constants** `n`

Whether it is 3n or 300n, doubling n doubles the time. Constants depend on the machine; growth does not.

3 **Keep the dominant term** `O(n)`

For $n^2 + 1000n$, at $n = 10^6$ the n^2 term is a thousand times bigger. The rest is noise.

Big-O describes how running time grows with input size - not how long it takes.

Your turn - call out the answer

A

```
int sum = 0;
for (int i = 0; i < n; i++)
    sum += a[i];
```

O(?)

B

```
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    if (a[i] + a[j] == k)
      count++;
```

O(?)

C

```
int steps = 0;
while (n > 1) {
    n = n / 2;
    steps++;
}
```

O(?)

Thirty seconds each. Say the answer and one sentence of why.

Your turn - call out the answer

A

```
int sum = 0;
for (int i = 0; i < n; i++)
    sum += a[i];
```

O(n)

One loop, n iterations, constant work inside. Linear.

B

```
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    if (a[i] + a[j] == k)
      count++;
```

O(n²)

For each of n values of i, the inner loop runs n times: n × n. Nested loops over the same range are quadratic.

C

```
int steps = 0;
while (n > 1) {
    n = n / 2;
    steps++;
}
```

O(log n)

n halves every step. How many halvings to reach 1? $\log_2 n$. Anything that halves - or doubles - is logarithmic.

Pattern: a loop over n is a factor of n. A nesting multiplies. A halving is a log.

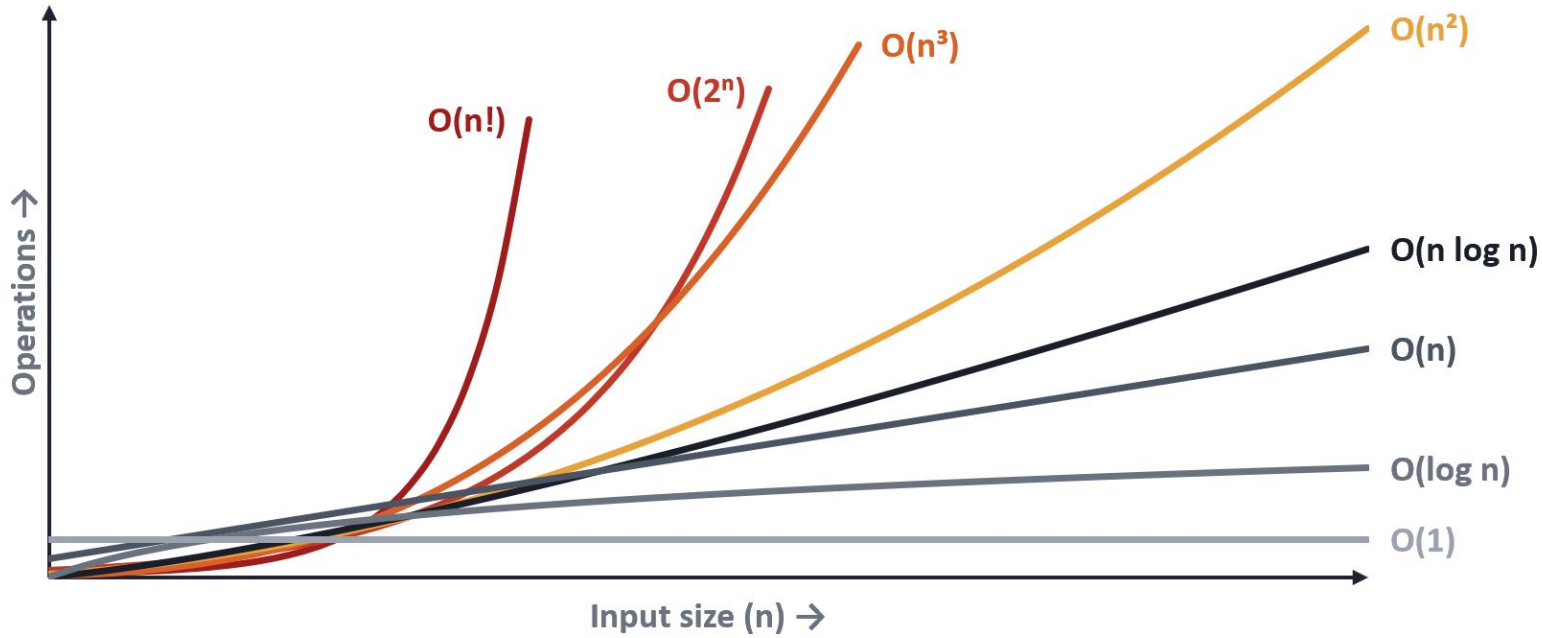
MEASURED ON A REAL MACHINE

The ladder, - and how big n can be

Class	Example used	n = 10	n = 11	n = 100	n = 1000
O(1)	constant_lookup — array index	117 ns	124 ns	124 ns	141 ns
O(log n)	binary_search — key absent	250 ns	245 ns	398 ns	602 ns
O(n)	linear_max — single pass scan	235 ns	252 ns	1.52 µs	13.5 µs
O(n log n)	merge_sort — recursive	4.82 µs	5.40 µs	75.0 µs	1.08 ms
$O(n^2)$	bubble_sort — no early exit	4.32 µs	3.46 µs	256 µs	32.7 ms
$O(n^3)$	count_zero_sum_triplets — 3-sum	7.08 µs	8.90 µs	5.20 ms	6.61 s
$O(2^n)$	count_subsets_with_sum — subsets	113 µs	223 µs	4.3×10^{15} yrs*	3.6×10^{286} yrs*
O(n!)	brute_force_tsp — permutations	2.27 s	23.6 s	1.7×10^{144} yrs*	7.5×10^{2553} yrs*

* Extrapolated from the measured rate, not actually run. Notice n! at n = 11 takes 11× longer than at n = 10: one extra city, ten times the wait.

How each class grows - operations against n



Same axes, same n. Constants shift a curve; only the class changes its shape - and the shape is what wins at large n.

Cheat sheet

LENGTH OF INPUT (n)		TARGET COMPLEXITY
$1 \le n \le 100$	→	$O(n^3)$ and lower · *triple nested loops are fine*
$100 < n \le 1{,}000$	→	$O(n^2)$ and lower · *one nested loop, no more*
$10^4 \le n \le 10^5$	→	$O(n \log n)$ and lower · *sort it, binary search, divide & conquer*
$n > 10^5$	→	$O(n)$ and lower · *single pass, hashing, two pointers*
$n > 10^9$	→	$O(\log n)$ and lower · *no room to read the input - use maths*

Examples that trip people up

```
for (int i = 0; i < n; i++)
  for (int j = i + 1; j < n; j++)
    check(a[i], a[j]);
```

Still $O(n^2)$

Inner loop runs n−1, n−2, … 1 times. Sum is n(n−1)/2. Drop constants: n^2. 'Half a square is still a square.'

```
for (int i = 0; i < n; i++) f(i);
for (int j = 0; j < m; j++) g(j);
```

$O(n + m)$, not $O(n \cdot m)$

Sequential loops add. Nested loops multiply. If m is tiny this is just O(n) - say so.

```
for (int i = 0; i < n; i++)
    binary_search(a, a + n, x[i]);
```

$O(n \log n)$

n iterations, each does a log n search. Multiply. Same class as sorting - which is a hint about how you might solve it differently.

ENOUGH TO NOT GET CAUGHT OUT

The notation, briefly

O(f)

Upper bound

Ω(f)

Lower bound

Θ(f)

Tight bound

Best, average, worst

Unless told otherwise, complexity means worst case. Quicksort is O(n log n) on average and O(n^2) in the worst case - if you say 'quicksort is n log n' without the qualifier, a good interviewer will ask you when it isn't. We cover this properly next week.

WORKED EXAMPLE

Quicksort - upper, lower and tight bounds

O Upper bound

$O(n^2)$

WORST CASE

Pivot is always the smallest or largest element (e.g. already-sorted input with a first-element pivot). Each partition peels off one item: $n + (n-1) + \ldots + 1 \approx n^2/2$.

Ω Lower bound

$\Omega(n \log n)$

BEST CASE

Pivot splits the array in half every time. log n levels of recursion, n work per level. No comparison sort can beat n log n.

Θ Tight bound

$\Theta(n \log n)$

AVERAGE CASE

With a random pivot the split is 'good enough' most of the time. Expected cost ≈ 1.39 n log n - the same class as the best case, so the bound is tight.

Read it like this: *Big-O says 'never slower than n^2'. Omega says 'never faster than n log n'. Theta says 'typically exactly n log n'. Same algorithm, three questions - and interviewers usually mean the worst case.*

MEMORY COUNTS TOO

Space complexity

```
int sum(int* a, int n) {
    if (n == 0) return 0;
    return a[n-1] + sum(a, n-1);
}
```

Time O(n). Space?

Also, O(n). Every recursive call sits on the stack until the base case. For n = 10^6 this crashes - stack overflow - even though it 'uses no extra memory'.

```
int sum(int* a, int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
```

Time O(n). Space O(1).

Two variables regardless of n. Same work, no stack growth. This is why interviewers ask 'can you do it iteratively?'

Rule: count the extra memory your algorithm allocates - arrays, maps, and the recursion stack. The input itself does not count.

SAME IDEA, SECOND EXAMPLE

Fibonacci - with and without extra space

```
long fib(int n) {
    if (n < 2) return n;
    return fib(n-1) + fib(n-2);
}
```

Time $O(2^n)$. Space O(n).

Every call spawns two more - the call tree has ~2^n nodes. The stack only holds one root-to-leaf path at a time, so space is the depth: n. fib(50) takes minutes; fib(10^5) overflows the stack.

```
long fib(int n) {
    long a = 0, b = 1;
    for (int i = 0; i < n; i++)
        b = a + b, a = b - a;
    return a;
}
```

Time O(n). Space O(1).

Two numbers carried forward, nothing remembered. Same answer, exponentially less work and no stack. A memo array would be O(n) time and O(n) space - the middle ground.

Notice: the naive version is bad on both axes. The fix for space (iterate) also fixed time - because the recursion was recomputing the same fib(k) over and over.

RULES OF THUMB

Cheat Sheet

1

Loops add, nesting multiplies

Two loops in sequence: $O(n + m)$. One inside the other: $O(n \cdot m)$.

2

Halving means log

Binary search, balanced trees, repeatedly dividing by two - all $\log n$.

3

Drop constants and lower terms

$3n^2 + 100n + 7$ is $O(n^2)$. Nobody cares about the 3.

4

Log base doesn't matter

$\log_2 n$ and $\log_{10} n$ differ by a constant.

5

Read the constraints first

$n \leq 10^5$ means n^2 is dead. That one sentence decides which algorithm you write.

6

Space includes the stack

Recursion depth is memory. Say it in the interview before they ask.

Before next session

1 Get your setup ready

Whatever setup you have chosen - VS Code, CLion, vim + gcc - make sure you can write, compile, execute and debug code on it (breakpoints, step, inspect).

2 Create your GitHub repo

One private repo for the course. Add me as a collaborator (Settings → Collaborators). Every submission lives there.

3 Fill the Google form

If you have not already, two minutes on the check-in form from earlier. It decides how fast we go.

4 Sorting & Searching: best, worst, average

Pick a few classics - bubble, insertion, selection, merge, quick sort; linear and binary search. For each, write the best, worst and average case and one line on why.

Push the complexity notes to your repo as **notes/complexity.md** -

06

Arrays (Backup)

If time allows

Arrays in memory: one contiguous block

```
int a[6] = {7, 3, 9, 1, 4, 8};
```

sizeof(int) = 4 bytes, so each cell is 4 bytes further along

7	3	9	1	4	8
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
0x1000	0x1004	0x1008	0x100C	0x1010	0x1014

$$\&a[i] = \text{base} + i \times \text{sizeof}(T)$$

std::vector<int>

Same block on the heap, plus three words of bookkeeping:

data	pointer to the block
size	elements in use
capacity	elements that fit before a regrow

Side Quest: Try to implement vector from scratch

(Use templates if possible)

2D and N-D arrays: still one block, laid out row by row

`int m[3][4];` how you picture it

0,0	0,1	0,2	0,3	row 0
1,0	1,1	**1,2**	1,3	row 1
2,0	2,1	2,2	2,3	row 2

flattened, row after row

how the hardware sees it: 12 ints, one after another

0,0	0,1	0,2	0,3	1,0	1,1	**1,2**	1,3	2,0	2,1	2,2	2,3
0	1	2	3	4	5	**6**	7	8	9	10	11

`&m[i][j] = base + (i·COLS + j)·size`

`m[1][2]` → $1 \cdot 4 + 2 = 6$ → base + 24

N-D: $idx = ((i_1 \cdot n_2 + i_2) \cdot n_3 + i_3) \ldots$

Peel one dimension at a time, innermost last. Row-major: C, C++, NumPy default. Fortran and MATLAB flip it (column-major).

Side Quest : how is vector<vector<int>> stored?

Write a sample code and inspect how and where it is stored.

Homework: Sum every element of a large matrix twice - once with `i` as the outer loop, once with `j` outer. Same work, same big-O. Time both. Which is faster, by how much, and what does the row-major picture above have to do with it?

Problems on arrays - the patterns behind them

1 Rotate by k, in place

Given an array of n integers and an integer k, rotate it right by k positions without using a second array.

Input: [1, 2, 3, 4, 5], k = 2

Output: [4, 5, 1, 2, 3]

Constraints: $1 \le n \le 10^5$, k may exceed n.

2 Range-sum queries

Given an array of n integers, answer q queries. Each query gives l and r; return the sum of a[l..r] inclusive.

Input: a = [2, 4, 1, 3], queries (0,2), (1,3)

Output: 7, 8

Constraints: $n, q \le 10^5$ - per-query loops will time out.

3 Dutch national flag

Given an array containing only 0s, 1s and 2s, sort it in a single pass without counting or using a sort library.

Input: [2, 0, 1, 2, 0, 1]

Output: [0, 0, 1, 1, 2, 2]

Constraints: one pass, O(1) extra space.

Try these before next session. Write the best, worst and average case for your solution - we will discuss approaches in class.

NEXT SESSION

Amortised analysis, recurrences, and why arrays beat linked lists in practice

Plus a live demo where the same loop runs ten times slower depending on the order of two nested for-loops. Bring a guess.

Cold-call topics: the growth ladder, the three examples, space complexity.

Questions?